Trevor Labat
Student ID: 801340975
Github: https://github.com/TlabatCharlotte/Machine-Learning

Problem 1 - Nonlinear Temperature Model

The temperature data from lecture is used with the input normalized the same way as lecture (t_un = 0.1 * t_u). The SGD optimizer from torch.optim and mean squared error loss are used for every training session.

Assumptions made: The parameters start at w2 = 1, w1 = 1, and b = 0, following the lecture starting values of w = 1 and b = 0. The linear baseline is the lecture model trained with a learning rate of 0.01 for 5000 epochs. No validation split was used since the lecture example trained on all 11 points.

1.a

The training loop from lecture 12 stays the same. Only the model function and the params tensor changed. params now holds 3 values [w2, w1, b] instead of 2, and since the loop calls model(t_u, *params) the extra value gets passed in automatically. Autograd finds the gradient for w2 so nothing had to be derived by hand. The code can be seen in Figure 1.1.

```
def model_nonlinear(t_u, w2, w1, b):
    return w2 * t_u**2 + w1 * t_u + b

def training_loop(n_epochs, optimizer, params, model, t_u, t_c):
    for epoch in range(1, n_epochs + 1):
        t_p = model(t_u, *params)
        loss = loss_fn(t_p, t_c)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        if epoch % 500 == 0:
            print(f"Epoch {epoch}, Loss {loss.item():.6f}")
    return params, loss.item()

results = {}
for lr in [1e-1, 1e-2, 1e-3, 1e-4]:
    print(f"\nLearning rate {lr}")
    params = torch.tensor([1.0, 1.0, 0.0], requires_grad=True)
    optimizer = optim.SGD([params], lr=lr)
    params, final_loss = training_loop(5000, optimizer, params, model_nonlinear, t_un, t_c)
    results[lr] = (params.detach().clone(), final_loss)
    print("Final params:", params.detach())
```

Figure 1.1 - Nonlinear model and training loop

1.b

Learning rates of 0.1, 0.01, 0.001, and 0.0001 were tested for 5000 epochs. The loss every 500 epochs is the following:

- lr = 0.1, 0.01, and 0.001
  - Loss was nan at every epoch from 500 to 5000
- lr = 0.0001
  - Epoch 500: 10.708596
  - Epoch 1000: 8.642083
  - Epoch 1500: 7.171005
  - Epoch 2000: 6.123478
  - Epoch 2500: 5.377227
  - Epoch 3000: 4.845286
  - Epoch 3500: 4.465788
  - Epoch 4000: 4.194724
  - Epoch 4500: 4.000802
  - Epoch 5000: 3.861744

The first three learning rates diverged and the loss became nan. Squaring the input makes the numbers a lot bigger so the gradients are bigger too, and the larger learning rates overshoot every step until the values overflow. Only 0.0001 was small enough to train. Final params: w2 = 0.5570, w1 = -0.8881, b = -0.8753.

1.c

The best nonlinear model is lr = 0.0001 since it was the only one that didn't diverge. Both models are plotted over the data in Figure 1.2.

- Nonlinear (lr = 0.0001) final loss: 3.8617
- Linear (lr = 0.01) final loss: 2.9276, with w = 5.3671 and b = -17.3012

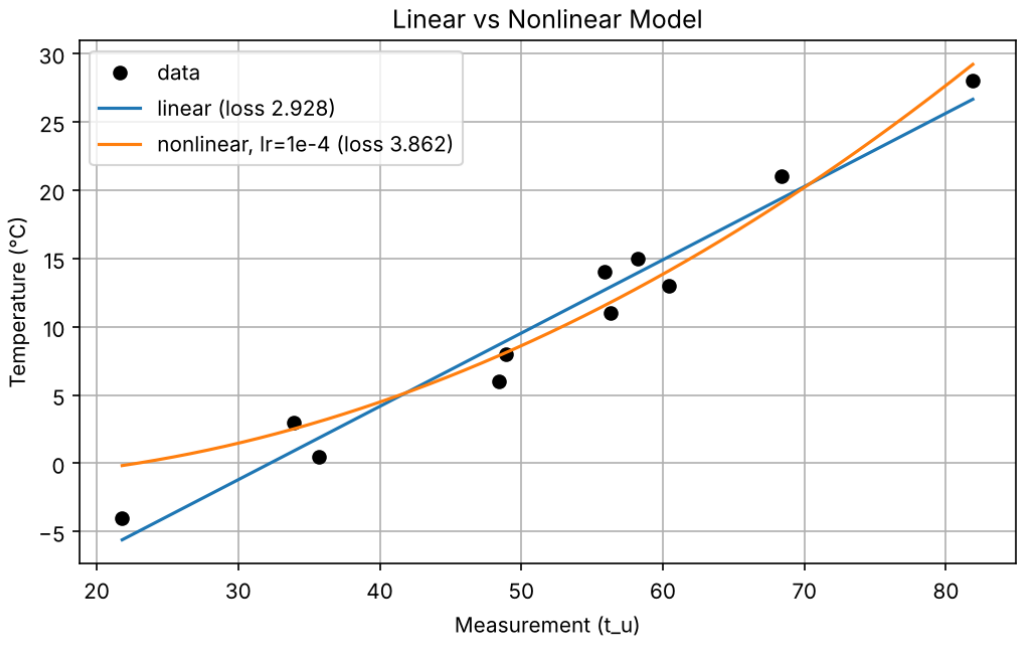


Figure 1.2 - Linear vs nonlinear model over the input data

The nonlinear result is worse than the linear baseline. A curve should be able to do at least as well as a line since setting w2 = 0 just gives back the linear model. The issue is the learning rate had to be so small to keep it from diverging that it didn't finish training in 5000 epochs. The loss was still going down at the end (4.0008 to 3.8617 over the last 1000 epochs), so with more epochs it would most likely catch up.

Problem 2 - Linear Regression for Housing Dataset (5 Features)

The housing data is loaded with pandas and area, bedrooms, bathrooms, stories, and parking are used to predict price. The data is split into an 80% training set and 20% validation set using torch.randperm like in lecture 13.

Assumptions made: torch.manual_seed(42) is set so the split stays the same. The inputs and price are standardized using the mean and standard deviation of the training data only and then applied to the validation data, to prevent information leakage. This means the loss values are in standardized units and not dollars. All parameters start at zero. Since this is regression and not classification, R2 on the validation set is used for validation accuracy.

2.a

There are six parameters, one weight for each input and a bias. The training loop is the one from lecture 13. Only the training loss calls backward, and the validation loss and R2 are found inside torch.no_grad() so the model never trains on the validation data. The code can be seen in Figure 2.1.

```
torch.manual_seed(42)
n = X.shape[0]
n_val = int(0.2 * n)
idx = torch.randperm(n)
train_idx, val_idx = idx[:-n_val], idx[-n_val:]
X_train, X_val = X[train_idx], X[val_idx]
y_train, y_val = y[train_idx], y[val_idx]

X_mean, X_std = X_train.mean(0), X_train.std(0)
y_mean, y_std = y_train.mean(), y_train.std()

X_train_n = (X_train - X_mean) / X_std
X_val_n   = (X_val   - X_mean) / X_std
y_train_n = (y_train - y_mean) / y_std
y_val_n   = (y_val   - y_mean) / y_std
def loss_fn(t_p, t_c):
    return ((t_p - t_c)**2).mean()
def model_multi(X, params):
    return X @ params[:-1] + params[-1]

def r2_score(y_pred, y_true):
    return 1 - ((y_true - y_pred)**2).sum() / ((y_true - y_true.mean())**2).sum()

def training_loop_val(n_epochs, optimizer, params, X_tr, y_tr, X_v, y_v):
    for epoch in range(1, n_epochs + 1):
        train_loss = loss_fn(model_multi(X_tr, params), y_tr)
        with torch.no_grad():
            val_pred = model_multi(X_v, params)
            val_loss = loss_fn(val_pred, y_v)
            val_r2 = r2_score(val_pred, y_v)
        optimizer.zero_grad()
        train_loss.backward()
        optimizer.step()
        if epoch % 500 == 0:
            print(f"Epoch {epoch}, Train loss {train_loss.item():.4f}, "
                  f"Val loss {val_loss.item():.4f}, Val R² {val_r2.item():.4f}")
    return params, val_loss.item()
```

Figure 2.1 - Preprocessing and training loop

2.b

Learning rates of 0.1, 0.01, 0.001, and 0.0001 were tested for 5000 epochs. The results every 500 epochs are the following (training loss / validation loss / validation R2):

- lr = 0.1
  - Epoch 500: 0.4335 / 0.3617 / 0.5298
  - Epoch 1000: 0.4335 / 0.3617 / 0.5298
  - Epoch 1500: 0.4335 / 0.3617 / 0.5298
  - Epoch 2000: 0.4335 / 0.3617 / 0.5298
  - Epoch 2500: 0.4335 / 0.3617 / 0.5298
  - Epoch 3000: 0.4335 / 0.3617 / 0.5298
  - Epoch 3500: 0.4335 / 0.3617 / 0.5298
  - Epoch 4000: 0.4335 / 0.3617 / 0.5298
  - Epoch 4500: 0.4335 / 0.3617 / 0.5298
  - Epoch 5000: 0.4335 / 0.3617 / 0.5298
- lr = 0.01
  - Epoch 500: 0.4335 / 0.3616 / 0.5299
  - Epoch 1000: 0.4335 / 0.3617 / 0.5298
  - Epoch 1500: 0.4335 / 0.3617 / 0.5298
  - Epoch 2000: 0.4335 / 0.3617 / 0.5298
  - Epoch 2500: 0.4335 / 0.3617 / 0.5298
  - Epoch 3000: 0.4335 / 0.3617 / 0.5298
  - Epoch 3500: 0.4335 / 0.3617 / 0.5298
  - Epoch 4000: 0.4335 / 0.3617 / 0.5298
  - Epoch 4500: 0.4335 / 0.3617 / 0.5298
  - Epoch 5000: 0.4335 / 0.3617 / 0.5298
- lr = 0.001
  - Epoch 500: 0.4572 / 0.3454 / 0.5509
  - Epoch 1000: 0.4373 / 0.3458 / 0.5504
  - Epoch 1500: 0.4345 / 0.3529 / 0.5411
  - Epoch 2000: 0.4338 / 0.3570 / 0.5358
  - Epoch 2500: 0.4336 / 0.3592 / 0.5330
  - Epoch 3000: 0.4335 / 0.3603 / 0.5315
  - Epoch 3500: 0.4335 / 0.3610 / 0.5307
  - Epoch 4000: 0.4335 / 0.3613 / 0.5303
  - Epoch 4500: 0.4335 / 0.3615 / 0.5300
  - Epoch 5000: 0.4335 / 0.3616 / 0.5299
- lr = 0.0001
  - Epoch 500: 0.8250 / 0.6305 / 0.1803
  - Epoch 1000: 0.7069 / 0.5356 / 0.3037
  - Epoch 1500: 0.6261 / 0.4710 / 0.3877
  - Epoch 2000: 0.5705 / 0.4271 / 0.4447

- Epoch 2500: 0.5321 / 0.3973 / 0.4834
- Epoch 3000: 0.5054 / 0.3773 / 0.5095
- Epoch 3500: 0.4867 / 0.3638 / 0.5270
- Epoch 4000: 0.4735 / 0.3549 / 0.5386
- Epoch 4500: 0.4640 / 0.3491 / 0.5462
- Epoch 5000: 0.4572 / 0.3455 / 0.5509

Final params (area, bedrooms, bathrooms, stories, parking, bias):

- lr = 0.1: [0.3942, 0.0297, 0.3010, 0.2481, 0.1749, 0]
- lr = 0.01: [0.3942, 0.0297, 0.3010, 0.2481, 0.1749, 0]
- lr = 0.001: [0.3941, 0.0301, 0.3008, 0.2478, 0.1751, 0]
- lr = 0.0001: [0.2826, 0.1053, 0.2355, 0.1895, 0.1702, 0]

0.1 and 0.01 reached the lowest training loss (0.4335) by epoch 500, and 0.001 got to the same params slower. 0.0001 was too slow and its training loss was still going down at epoch 5000.

0.0001 has a slightly better validation loss (0.3455) than the others (0.3617), but that is only because it stopped before fully fitting the training data and happened to do a little better on this split. With only 109 houses in validation this could flip with a different split. lr = 0.1 was picked as the best model since it converged the fastest, with a validation R2 of 0.5298.

Since the inputs are standardized the weights can be compared to each other. Area affects price the most, then bathrooms, stories, and parking. Bedrooms barely matters. The bias is basically 0 because the data was centered at 0.

2.c

HW 2 used the same 5 variables standardized, a learning rate of 0.01, 2000 iterations, and lambda = 1. It split the data with train_test_split using random_state = 100, and its cost was half of the MSE in dollars. The HW 2 results were the following:

- Final training cost: 7.61×10^11
- Final validation cost: 7.98×10^11
- Validation R2: 0.5477
- Final theta: [4795729.21, 729929.12, 79339.82, 641260.97, 463851.58, 287330.53]

The losses can't be compared directly since HW 2 left price in dollars, but R2 can. HW 2 got 0.5477 and HW 5 got 0.5298. The weights can also be compared by dividing each one by the area weight:

- area: HW 2 = 1.000, HW 5 = 1.000
- bedrooms: HW 2 = 0.109, HW 5 = 0.075
- bathrooms: HW 2 = 0.879, HW 5 = 0.764
- stories: HW 2 = 0.635, HW 5 = 0.629
- parking: HW 2 = 0.394, HW 5 = 0.444

There isn't a meaningful difference. Both homeworks fit the same linear model with gradient descent. HW 2 did the gradient by hand in numpy and HW 5 used autograd and the SGD optimizer. Both rank the features in the same order. The small differences come from the different data split, the small regularization in HW 2, and the different loss units.

Problem 3 - Linear Regression for Housing Dataset (All Features)

Problem 2 is repeated using every input feature in the housing data.

Assumptions made: The yes/no columns (mainroad, guestroom, basement, hotwaterheating, airconditioning, prefarea) were mapped to ones and zeros. furnishingstatus has 3 options, so it was split into 2 columns (semi-furnished and unfurnished) with furnished as the baseline. This gives 13 features and 14 parameters. The same split, standardization, starting values, and R2 from Problem 2 were used.

3.a

The model and training loop are the same as Problem 2. Only the preprocessing changed, which can be seen in Figure 3.1.

```
housing3 = housing.copy()
binary_cols = ['mainroad', 'guestroom', 'basement', 'hotwaterheating', 'airconditioning', 'prefarea']
for c in binary_cols:
    housing3[c] = housing3[c].map({'yes': 1, 'no': 0})

housing3 = pd.get_dummies(housing3, columns=['furnishingstatus'], drop_first=True, dtype=float)

features3 = [c for c in housing3.columns if c != 'price']
print(features3, len(features3))

X3 = torch.tensor(housing3[features3].values, dtype=torch.float32)

X3_train, X3_val = X3[train_idx], X3[val_idx]
X3_mean, X3_std = X3_train.mean(0), X3_train.std(0)
X3_train_n = (X3_train - X3_mean) / X3_std
X3_val_n   = (X3_val   - X3_mean) / X3_std
```

Figure 3.1 - Preprocessing for all features

3.b

The results every 500 epochs are the following (training loss / validation loss / validation R2):

- lr = 0.1
  - Epoch 500: 0.3188 / 0.2527 / 0.6715
  - Epoch 1000: 0.3188 / 0.2527 / 0.6715
  - Epoch 1500: 0.3188 / 0.2527 / 0.6715
  - Epoch 2000: 0.3188 / 0.2527 / 0.6715
  - Epoch 2500: 0.3188 / 0.2527 / 0.6715
  - Epoch 3000: 0.3188 / 0.2527 / 0.6715
  - Epoch 3500: 0.3188 / 0.2527 / 0.6715
  - Epoch 4000: 0.3188 / 0.2527 / 0.6715

    - Epoch 4500: 0.3188 / 0.2527 / 0.6715
    - Epoch 5000: 0.3188 / 0.2527 / 0.6715
- lr = 0.01
    - Epoch 500: 0.3188 / 0.2521 / 0.6722
    - Epoch 1000: 0.3188 / 0.2526 / 0.6715
    - Epoch 1500: 0.3188 / 0.2527 / 0.6715
    - Epoch 2000: 0.3188 / 0.2527 / 0.6715
    - Epoch 2500: 0.3188 / 0.2527 / 0.6715
    - Epoch 3000: 0.3188 / 0.2527 / 0.6715
    - Epoch 3500: 0.3188 / 0.2527 / 0.6715
    - Epoch 4000: 0.3188 / 0.2527 / 0.6715
    - Epoch 4500: 0.3188 / 0.2527 / 0.6715
    - Epoch 5000: 0.3188 / 0.2527 / 0.6715
- lr = 0.001
    - Epoch 500: 0.3341 / 0.2396 / 0.6885
    - Epoch 1000: 0.3222 / 0.2405 / 0.6873
    - Epoch 1500: 0.3200 / 0.2440 / 0.6828
    - Epoch 2000: 0.3192 / 0.2467 / 0.6793
    - Epoch 2500: 0.3190 / 0.2486 / 0.6767
    - Epoch 3000: 0.3189 / 0.2500 / 0.6750
    - Epoch 3500: 0.3188 / 0.2509 / 0.6738
    - Epoch 4000: 0.3188 / 0.2515 / 0.6731
    - Epoch 4500: 0.3188 / 0.2519 / 0.6726
    - Epoch 5000: 0.3188 / 0.2521 / 0.6722
- lr = 0.0001
    - Epoch 500: 0.7427 / 0.5567 / 0.2762
    - Epoch 1000: 0.5860 / 0.4280 / 0.4435
    - Epoch 1500: 0.4895 / 0.3506 / 0.5442
    - Epoch 2000: 0.4298 / 0.3041 / 0.6046
    - Epoch 2500: 0.3927 / 0.2764 / 0.6407
    - Epoch 3000: 0.3693 / 0.2599 / 0.6621
    - Epoch 3500: 0.3544 / 0.2502 / 0.6748
    - Epoch 4000: 0.3448 / 0.2445 / 0.6821
    - Epoch 4500: 0.3384 / 0.2413 / 0.6863
    - Epoch 5000: 0.3341 / 0.2396 / 0.6885

Final weights for lr = 0.1:

- area: 0.2930
- bathrooms: 0.2588
- stories: 0.2215
- airconditioning: 0.1962
- prefarea: 0.1628

- parking: 0.1332
- basement: 0.0930
- hotwaterheating: 0.0666
- mainroad: 0.0591
- guestroom: 0.0495
- bedrooms: 0.0160
- semi-furnished: -0.0313
- unfurnished: -0.1264
- bias: 0.0000

Same pattern as Problem 2. 0.1 and 0.01 reached the lowest training loss (0.3188) quickly, 0.001 got there slower, and 0.0001 didn't finish. 0.0001 again has a slightly higher validation R2 (0.6885) for the same reason as before, so lr = 0.1 was picked as the best model with a validation R2 of 0.6715.

Using all the features made the model better. Validation R2 went up from 0.5298 to 0.6715 and training loss went down from 0.4335 to 0.3188. Area, bathrooms, and stories are still the biggest, but air conditioning and preferred area also matter a lot. Being unfurnished lowers the price compared to furnished.

3.c

HW 2 used 11 variables since it didn't include furnishingstatus. The results compared with HW 5 are the following:

- HW 2 (5 features): validation R2 = 0.5477
- HW 5 Problem 2 (5 features): validation R2 = 0.5298
- HW 2 (11 features): validation R2 = 0.6459, validation cost = 6.25×10^11
- HW 5 Problem 3 (13 features): validation R2 = 0.6715

There isn't a meaningful difference. HW 5 is about 0.026 higher, which is small enough to come from the different data split. Both are the same linear model trained with gradient descent, HW 5 just uses autograd and the SGD optimizer instead of the hand written gradient. The small difference comes from the different split, HW 5 having the 2 furnishingstatus columns, and the regularization in HW 2. The weights weren't compared because HW 2 only standardized the 5 number columns and left price in dollars, so the weights are on different scales.